

AI-103 Study Guide: Module 1.4 Enhanced

Develop Generative AI Apps That Use Tools

This is an ENHANCED version with critical function tool execution emphasis

1. Exam Domain Mapping

AI-103 Exam Domain	Relevance	Why
Implement generative AI and agentic solutions	High	Covers tool calling with OpenAI Responses API: <code>code_interpreter</code> , <code>web_search</code> , <code>file_search</code> , and <code>function</code> tools
Plan and manage an Azure AI solution	Medium–High	Choosing the right tool, endpoint, authentication, grounding method, and safety controls
Implement information extraction solutions	Medium	<code>file_search</code> and vector stores ground responses in uploaded documents
Implement text analysis solutions	Low–Medium	Tools can support text processing, summarization, policy Q&A
Implement computer vision solutions	Low	Not the main focus, though tools can be combined with multimodal scenarios

Exam focus: How tools extend a generative AI model beyond basic chat by allowing it to retrieve information, run code, search files, search the web, or call application functions.

2. What This Module Teaches

This module teaches how to build generative AI applications that use tools through the **OpenAI Responses API**.

Key idea:

A model without tools can reason and generate text. A model with tools can retrieve current information, ground responses in files, run code, call business functions, and support more practical workflows.

CRITICAL DISTINCTION: Tools in a Responses API request are **NOT the same as Foundry Tools**.

* **Responses API tools** = `code_interpreter`, `web_search`, `file_search`, `function` (specified in the prompt request)

* **Foundry Tools** = Azure AI APIs such as Language, Speech, Translator, Document Intelligence, Content Understanding

3. Key Concepts

3.1 Tool Types and Selection

Tool	What It Does	Best Use Case	Key Limitation
<code>code_interpreter</code>	Runs Python code in a sandboxed environment	Calculations, data analysis, file processing, simulations	No external network access from sandbox
<code>web_search</code>	Searches the public web at runtime	Current events, recent facts, product updates, policies	Source quality varies, results change over time
<code>file_search</code>	Searches uploaded files indexed in a vector store	Internal documents, policy Q&A, manuals, contracts, support knowledge	Retrieval quality depends on document quality and chunking
<code>function</code>	Requests developer-defined function calls	APIs, databases, workflows, business logic, system integration	APPLICATION must execute the function

3.2 Tool Configuration Pattern

Tools are provided in the `tools` array of a `responses.create()` call:

```
response = client.responses.create(  
    model=model_deployment,  
    instructions="You are a helpful AI assistant.",  
    input="Find information about vintage computers.",  
    tools=[  
        {  
            "type": "web_search"  
        },  
        {  
            "type": "file_search",  
            "vector_store_ids": [vector_store.id]  
        },  
        {  
            "type": "code_interpreter",
```

```
        "container": {"type": "auto"}
    }
]
)
```

Model's Role: The model decides WHETHER to use a tool and WHICH tool to use, based on the prompt.

Application's Role:

- * Specify which tools are available
 - * Control which data sources are connected
 - * Execute function calls
 - * Handle results
 - * Validate outputs
-

3.3 code_interpreter Tool

Configuration

```
tools=[
    {
        "type": "code_interpreter",
        "container": {"type": "auto"}
    }
]
```

How It Works

1. Model writes Python code
2. Code runs in sandboxed environment
3. Execution result returned to model
4. Model explains final result

Pre-installed Libraries

- * pandas
- * numpy
- * matplotlib
- * math
- * Standard Python libraries

Limitations

- * **No external network access** - Cannot browse internet from the sandbox
- * **Timeout limits** - Long-running operations may timeout
- * **Memory constraints** - Large datasets may need streaming or chunking
- * **Sandbox isolation** - Cannot access system files or resources

Example

```
response = client.responses.create(
    model="gpt-4.1",
    instructions="Use the python tool to solve math problems.",
```

```
input="What is the square root of 16?",
tools=[{"type": "code_interpreter", "container": {"type": "auto"}}]
)
# Model generates and runs: import math; math.sqrt(16)
```

3.4 web_search Tool

Configuration

```
tools=[
    {"type": "web_search"}
]
```

How It Works

1. Model evaluates whether fresh data is needed
2. Model issues search query
3. Results retrieved and reviewed
4. Model generates source-aware answer

Best for

- * Current events, breaking news
- * Recent product releases
- * Policy updates
- * Market research
- * Fact verification

Limitations

- * **Source quality varies** - Not all results are reliable
 - * **Results change over time** - Same query on different days = different results
 - * **Public information only** - Cannot access paywalled or private content
 - * **Latency impact** - Web retrieval adds to response time
-

3.5 file_search Tool

Configuration and Usage

```
# Step 1: Create and populate vector store
vector_store = client.vector_stores.create(name="policy-docs")
client.vector_stores.files.upload_and_poll(
    vector_store_id=vector_store.id,
    file=open("expenses_policy.pdf", "rb")
)

# Step 2: Use file_search in responses
response = client.responses.create(
    model="gpt-4.1",
    instructions="Answer using the uploaded policy documents.",
    input="What is the maximum taxi claim amount?",
```

```
tools=[
    {
        "type": "file_search",
        "vector_store_ids": [vector_store.id]
    }
],
include=["file_search_call.results"] # For debugging
)
```

How It Works

1. Files uploaded and indexed into vector store
2. User question converted to embedding
3. Semantic search retrieves relevant chunks
4. Matching passages injected into prompt context
5. Model answers using retrieved content

Vector Store Management

- * Create: `client.vector_stores.create(name="...")`
- * Upload: `client.vector_stores.files.upload_and_poll(...)`
- * Search happens automatically when `file_search` tool is used

Retrieval Quality Depends On

- * Document quality and clarity
- * Chunking strategy and size
- * Vector store scope (separate by domain)
- * Updated files may need re-indexing

3.6 function Tool (CRITICAL - APPLICATION RESPONSIBILITY)

■■ MOST IMPORTANT POINT FOR AI-103 EXAM

THE MODEL DOES NOT RUN YOUR FUNCTION AUTOMATICALLY.

The execution flow is:

1. **Model emits** `function_call` (with function name and arguments)
2. **Application validates** the function call
3. **Application runs** the matching function
4. **Application returns** `function_call_output` with result
5. **Model incorporates** the result into the final response

Configuration and Implementation

```
function_tools = [
    {
        "type": "function",
        "name": "get_time",
        "description": "Get the current time"
    }
]
```

```

def get_time():
    """Function implementation"""
    return f"The time is {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime())}"

# Initial request
response = client.responses.create(
    model=model_deployment,
    input=[
        {"role": "developer", "content": "You are an AI assistant."},
        {"role": "user", "content": "What time is it?"}
    ],
    tools=function_tools
)

# Check if model requested a function call
for item in response.output:
    if item.type == "function_call" and item.name == "get_time":
        # ■ APPLICATION RUNS THE FUNCTION
        current_time = get_time()

        # ■ APPLICATION RETURNS THE RESULT
        messages.append({
            "type": "function_call_output",
            "call_id": item.call_id,
            "output": current_time
        })

        # ■ GET FOLLOW-UP RESPONSE
        response = client.responses.create(
            model=model_deployment,
            input=messages,
            tools=function_tools
        )

```

Validation Requirements

WARN CRITICAL: Always validate function arguments before execution:

```

if item.type == "function_call":
    # VALIDATE inputs
    if item.name == "transfer_money":
        # Check: Is amount reasonable?
        if item.arguments.get("amount", 0) > 10000:
            # Require approval or reject
            pass

    # VALIDATE permissions
    if item.name == "delete_user":
        # Check: Does user have permission?
        pass

# Run function only after validation
result = execute_function(item.name, item.arguments)

```

Common Use Cases

- * Querying databases for information
- * Calling internal APIs
- * Triggering workflows
- * Executing business logic
- * Integrating with external systems

4. Tool Limitations and Best Practices

code_interpreter

- * ■ Good for: math, data analysis, file processing
- * ■ Bad for: network-dependent tasks, long-running operations
- * **Sandbox Limitations:** No network, timeout, memory constraints
- * **Best Practice:** Review generated code before production use

web_search

- * ■ Good for: current facts, recent events
- * ■ Bad for: reliable sources only, long-term consistency
- * **Source Quality:** Varies significantly
- * **Best Practice:** Verify critical facts from returned sources

file_search

- * ■ Good for: internal documents, proprietary knowledge
- * ■ Bad for: huge datasets, mixed domains
- * **Chunking Matters:** Poor chunking = poor retrieval
- * **Best Practice:** Scope vector stores by domain; update regularly

function

- * ■ Good for: system integration, business logic
- * ■ Bad for: unvalidated high-impact actions
- * **Security Critical:** Application controls execution
- * **Best Practice:** Validate inputs, check permissions, log calls

5. Azure Services / Tools

Service / Tool	Role	Exam Relevance
Microsoft Foundry Models	Provides models that can support tool calling	High

Service / Tool	Role	Exam Relevance
OpenAI Responses API	Main API used to specify tools in model requests	High
code_interpreter tool	Runs Python code in a sandbox	High
web_search tool	Retrieves current public web information	High
file_search tool	Retrieves information from uploaded indexed files	High
Vector stores	Store indexed files for file_search	High
function tool	Allows custom application functions to be called	High
OpenAI SDK	Used in the lab to call the model	High
Azure OpenAI endpoint	Endpoint used for OpenAI-compatible model access	High
Microsoft Entra ID	Preferred authentication method	High
Foundry IQ	Recommended for enterprise-scale agent knowledge stores	Medium–High

6. Decision Rules

Scenario	Best Choice	Why
Need calculations, statistics, data transformation	code_interpreter	Runs Python in sandbox
Need current information from internet	web_search	Retrieves fresh public data
Need answers from uploaded private documents	file_search	Searches indexed file content
Need to call internal APIs, databases, workflows	function tool	Application executes controlled logic

Scenario	Best Choice	Why
Need enterprise-scale knowledge across many stores	Foundry IQ with agents	More scalable than simple file search
Need real-time current facts	web_search	Training data may be outdated
Need authoritative internal policy answers	file_search	Ground answers in controlled source docs
Need high-impact actions (orders, emails, payments)	Function tool with validation	Application must control and validate
Need deterministic business logic	Function tool	Do not rely on model to infer rules
Need model to choose between multiple tools	Provide multiple tools in array	Model selects based on prompt
Need safer production behavior	Validate all inputs and outputs	Tool outputs can affect real systems
Need production authentication	Microsoft Entra ID	Preferred over API keys

7. Exam Traps

Trap	Correct Understanding
Confusing Responses API tools with Foundry Tools	Responses API tools = code_interpreter, web_search, file_search, function. Foundry Tools = Language, Speech, Translator, Document Intelligence
Assuming model can access current facts by default	Needs web_search or another external data source
Assuming model can answer from private docs automatically	Needs file_search with uploaded/indexed files in vector store
Thinking file_search is the same as web search	file_search searches YOUR indexed files; web_search searches public web
Thinking the model runs custom functions itself	Your application runs the function and returns function_call_output
Trusting function arguments without validation	Always validate inputs and permissions before executing
Using function calling for high-impact actions without controls	Use authorization, approval gates, logging, human review
Assuming code_interpreter can browse the web	Runs Python in sandbox with NO external network access

Trap	Correct Understanding
Assuming <code>code_interpreter</code> results are always correct	Review generated code and results, especially in production
Assuming web results are always reliable	Source quality varies; requires validation
Using one huge vector store for unrelated domains	Scope carefully to improve retrieval precision
Forgetting to re-index updated files	Updated content may not be searchable until re-uploaded
Thinking tool use has no cost/performance impact	Tool calls increase latency, token usage, and cost
Using API keys casually in production	Prefer Entra ID; store keys securely

8. Quick Revision Questions

1. Why do generative AI models need tools?
2. What is the difference between Responses API tools and Foundry Tools?
3. Which tool should you use for calculations and data analysis?
4. Which tool should you use for current public information?
5. Which tool should you use to answer from uploaded private documents?
6. Which tool should you use to call an internal business API?
7. Where are tools specified in a Responses API request?
8. Does the model always use every tool listed in the `tools` array?
9. What does the `code_interpreter` tool run?
10. What are the limitations of the `code_interpreter` sandbox?
11. Why does `web_search` reduce hallucination risk?
12. Why can web search still require human validation?
13. What is a vector store used for?
14. What does `file_search` retrieve?
15. Why might retrieval quality be poor in a large mixed-domain vector store?
16. **What should your app do after the model returns a `function_call` item?** (Answer: Run the function and return `function_call_output`)
17. **Does the model run custom function code automatically?** (Answer: NO - your application must run it)
18. Why must function arguments be validated?
19. What should be logged for tool governance?
20. Why can tools increase response latency?
21. In the lab, which endpoint should be used: project or Azure OpenAI?
22. Why is Entra ID preferred for production authentication?
23. When should you consider Foundry IQ instead of simple `file_search`?

Final Exam Memory

MOST CRITICAL DISTINCTION FOR AI-103:

- WRONG: Model executes the function
- RIGHT: Model requests function → App executes → App returns result → Model answers

Remember the tool-selection shortcut:

Need code/math/data?	→ code_interpreter
Need current info?	→ web_search
Need private docs?	→ file_search
Need app/API/business action?	→ function

Remember the execution pattern:

```
Model emits function_call
↓
Application validates call
↓
Application runs function
↓
Application sends function_call_output
↓
Model generates final response
```

Key Enhancements in This Version

- OK Added critical function tool execution responsibility emphasis
- OK Added function implementation code pattern with validation
- OK Added tool configuration syntax for each tool
- OK Added vector store creation and upload patterns
- OK Added pre-installed library list for code_interpreter
- OK Added sandbox limitation specifics
- OK Added function validation requirements
- OK Added execution flow diagram
- OK Clarified critical distinction from Foundry Tools